TESTAREA SISTEMELOR SOFTWARE (TSS) • PROIECT DE LABORATOR

Smart Corporate Multi-Tenant Parking Gate

Verificare Funcțională, Analiză Structurală MC/DC și Mutation Testing pe Motorul de Decizie

Echipa de Proiect (Contribuții 50% - 50%):

- Victor Mura (50%): Testare Funcțională (EP, BVA, CEG), Analiză Coverage Separat, Mutanți Manuali Ne-Echivalenți
- Natalia Moroz (50%): Analiză Structurală (CFG, V(G)=7, Căi Liniar Independente), MC/DC, Mutant Echivalent, Mutation Tools

Universitatea din București • Master Ingineria Sistemelor Software (Anul II)

Responsabil: Echipa (Victor & Natalia)

Contextul Problemei & Specificația Programului

🏢 Scenariul de Business Multi-Tenant

• Un parc de birouri modern găzduiește multiple companii chiriașe cu cote contractuale fixe de locuri gratuite.
• Bariera inteligentă scanează vehiculul și decide accesul instant pe baza a 7 factori deterministi.
• Infrastructura dispune de stații comune de încărcare electrică (EV) și o zonă de overflow contra cost.
• Metoda centrală testată: evaluate_parking_access(...) implementată pur în Python 3.11, fără I/O nedeterminist.

⚙️ Ierarhia Regulilor Decizionale

R0. Validare Defensivă: Aruncă ValueError la ore < 0 sau > 23, cote <= 0, locuri ocupate negative.
R1. Protocol VIP: Acces garantat necondiționat pe loc dedicat (GRANTED_VIP).
R2. Securitate Contractor: Acces permis strict între orele 08:00 - 17:59 (DENIED_RESTRICTED_HOURS altfel).
R3. Stații Încărcare EV: Condiție compusă: Mașină EV + Stație liberă + (Off-peak SAU în cota firmei).
R4. Cotă Gratuită: Locuri ocupate < Cota firmei -> GRANTED_REGULAR.
R5. Clauză Overflow: Peste cotă, dar firma plătește overflow (noaptea oricât, ziua < 5 locuri depășire).
R6. Respingere: Cotă epuizată și fără overflow -> DENIED_QUOTA_EXCEEDED.

Responsabil: Victor Mura (20%)

Cerința 1 (a & b): Equivalence Partitioning & Boundary Value Analysis

📊 Equivalence Partitioning (13 Teste)

- Cota firmei (q): Valid [1, inf) | Invalide q = 0, q < 0 (ValueError).
- Locuri ocupate (o): Sub cotă (o < q), La cotă (o = q), Peste cotă (o > q) | Invalid o < 0.
- Ora sosirii (h): Off-peak dimineață [0, 7], Peak hours [8, 17], Off-peak seară [18, 23] | Invalide h < 0, h >= 24.
- Rol angajat: REGULAR, VIP (prioritar), CONTRACTOR (restricționat orar).
- Opțiuni booleene: EV solicitat/disponibil, politică overflow activă/inactivă.
- Implementare: tests/test_ep.py — acoperire completă a tuturor claselor valide și invalide.

🎯 Boundary Value Analysis (24 Teste)

- Frontiere Cotă: 0 (invalid), 1 (min valid), 2 (min + 1).
- Frontiere Ocupat: -1 (invalid), 0 (min valid), 1 (min + 1).
- Frontiere Relaționale Cotă (q = 10): o = 9 (sub cotă -> Regular), o = 10 (la cotă -> Denied), o = 11 (peste cotă).
- Frontiere Orar Extrem: -1 (invalid), 0, 1 și 22, 23, 24 (invalid).
- Frontiere Interval Vârf: 7 (off-peak), 8 (start vârf), 9; 16, 17 (sfârșit vârf), 18 (off-peak).
- Prag Critic Overflow Ziua: diferență = 4 (permis), 5 (respins - limita strictă < 5), 6 (respins).
- Implementare: tests/test_bva.py — detecție off-by-one.

Responsabil: Victor Mura (15%)

Cerința 1.c: Cause-Effect Graphing (CEG) & Tabel de Decizie

Graful Cauză-Efect: Cauze & Porți Logice

- C1: tier == VIP | C2: tier == CONTRACTOR (Exclusivitate E(C1, C2))
- C3: is_peak_hours (08:00 - 17:59)
- C4: is_electric_vehicle | C5: has_ev_spots_available
- C6: is_within_quota (occupied < quota)
- C7: company_allows_overflow | C8: overflow_count < 5
- Noduri Intermediare (Porți AND / OR / NOT):
- I1 = NOT(C3) (sosire în afara orelor de vârf)
- I2 = C2 AND I1 (contractor noaptea -> respins orar)
- I3 = C4 AND C5 (vehicul EV cu încărcător liber)
- I4 = I1 OR C6 (condiție relaxată EV: noaptea sau în cotă)
- I5 = I3 AND I4 -> Acces la stația EV
- I7 = C7 AND (I1 OR C8) -> Acces aprobat la Overflow

Tabelul de Decizie Derivat (9 Reguli)

R1: C1=1 -> E1: GRANTED_VIP (fără condiționări suplimentare).
R2: C1=0, C2=1, C3=0 -> E2: DENIED_RESTRICTED_HOURS.
R3: Non-VIP, C3=0 (off-peak), C4=1, C5=1 -> E3: GRANTED_EV_CHARGING.
R4: Non-VIP, C3=1 (peak), C4=1, C5=1, C6=1 -> E3: GRANTED_EV_CHARGING.
R5: Non-VIP, Non-EV, C6=1 (în cotă) -> E4: GRANTED_REGULAR.
R6: Peste cotă, C7=1 (overflow), C3=0 (off-peak) -> E5: GRANTED_PAID_OVERFLOW.
R7: Peste cotă, C7=1, C3=1 (peak), C8=1 (< 5) -> E5: GRANTED_PAID_OVERFLOW.
R8: Peste cotă, C7=1, C3=1, C8=0 (>= 5) -> E6: DENIED_QUOTA_EXCEEDED.
R9: Peste cotă, C7=0 (fără overflow) -> E6: DENIED_QUOTA_EXCEEDED.

Responsabil: Victor Mura (15%)

Cerința 2: Analiză Comparativă a Acoperirii de Cod

📈 Rezultate Obținute Separat (pytest-cov --cov-branch pe src/parking_gate.py)

- **Equivalence Partitioning (13 teste): 29/29 Linii (100%) • 12/12 Ramuri (100%) • Acoperire Globală: 100%**
- **Boundary Value Analysis (24 teste): 27/29 Linii (93%) • 10/12 Ramuri (83%) • Acoperire Globală: 90%**
- **Cause-Effect Graphing (9 teste): 28/29 Linii (96%) • 11/12 Ramuri (91%) • Acoperire Globală: 95%**

💡 Comentarii & Interpretarea Calitativă a Rezultatelor

- EP (100%): Eficiență maximă de ramificare structurală. Forțează parcurgerea fiecărei categorii valide și invalide (ValueError, VIP, EV, Overflow). Limitare: nu garantează corectitudinea la limita exactă a intervalului.
- BVA (90%): Se concentrează exclusiv pe variabile continue/ordonate (numere). Neacoperirea liniei 61 (VIP) și a ramurii EV provine din faptul că VIP și EV sunt decizii calitative discrete, nu frontiere numerice.
- CEG (95%): Cu doar 9 cazuri de test (cel mai compact set), acoperă aproape toate ramurile de business. Singura linie neacoperită este linia 54 (raise ValueError), deoarece CEG a modelat doar combinațiile operaționale valide ale parcului.
- Concluzie Metodologică: Tehnicile sunt strict complementare — BVA previne erorile de tip off-by-one, CEG optimizează deciziile logice compuse, iar EP garantează că niciun domeniu de date nu este neglijat.

Responsabil: Natalia Moroz (15%)

Cerința 3: Graful de Control (CFG) și Complexitatea Ciclomatică

Structura Grafului CFG & Calculul V(G)

Graful orientat are N = 14 noduri de bază și E = 19 arce decizionale conectate la nodul terminal N14 (Exit).

Calculul Complexității Ciclomatice prin 3 Metode:

- 1. Formula muchii & noduri: V(G) = E - N + 2P = 19 - 14 + 2(1) = 7
- 2. Formula noduri predicat: V(G) = P_predicat + 1 = 6 + 1 = 7
  (Nodurile predicat cu >= 2 ieșiri sunt: N1, N3, N5, N7, N9, N11)
- 3. Formula regiunilor plane: V(G) = R = 6 regiuni închise + 1 deschisă = 7
- Concluzie Matematică: Toate cele 3 formalisme converg riguros către V(G) = 7 circuite independente.

Baza celor 7 Căi Liniar Independente

- Path 1: N1 -> N2 -> N14 (Eroare Validare: q <= 0 -> ValueError)
- Path 2: N1 -> N3 -> N4 -> N14 (VIP -> GRANTED_VIP)
- Path 3: N1 -> N3 -> N5 -> N6 -> N14 (Contractor noaptea -> DENIED_RESTRICTED_HOURS)
- Path 4: N1 -> N3 -> N5 -> N7 -> N8 -> N14 (EV aprobat -> GRANTED_EV_CHARGING)
- Path 5: N1 -> N3 -> N5 -> N7 -> N9 -> N10 -> N14 (În cotă -> GRANTED_REGULAR)
- Path 6: N1 -> N3 -> N5 -> N7 -> N9 -> N11 -> N12 -> N14 (Overflow aprobat -> GRANTED_PAID_OVERFLOW)
- Path 7: N1 -> N3 -> N5 -> N7 -> N9 -> N11 -> N13 -> N14 (Cotă plină -> DENIED_QUOTA_EXCEEDED)

Acest set acoperă 100% din muchiile grafului.

Responsabil: Natalia Moroz (15%)

Cerința 4: Modified Condition / Decision Coverage (MC/DC)

🎯 Decizia Compusă & Standardul DO-178C

• Criteriul MC/DC (standardul aeronautic DO-178C Level A) impune ca fiecare condiție atomică să influențeze decizia independent de celelalte.
• Pentru N = 4 condiții, numărul minim teoretic este N + 1 = 5 teste.
• Decizia analizată (Regula 3 - Stații EV):
• Decision_EV = (A and B) and (not C or D)
• unde:
• A = is_electric_vehicle
• B = has_ev_spots_available
• C = is_peak_hours (not C = off-peak)
• D = is_within_quota (occupied < quota)

🧪 Perechi de Independență & Set Minimal (5 Teste)

T1: (1, 1, 0, 0) -> Decision = True
Bază Adevărată (EV, Spot liber, Off-peak, Peste cotă)
T2: (0, 1, 0, 0) -> Decision = False
Pereche Independență A: (T1, T2) -> Doar A comută 1->0, forțând decizia 1->0.
T3: (1, 0, 0, 0) -> Decision = False
Pereche Independență B: (T1, T3) -> Doar B comută 1->0, forțând decizia 1->0.
T4: (1, 1, 1, 0) -> Decision = False
Pereche Independență C: (T1, T4) -> Doar C comută 0->1, forțând decizia 1->0.
T5: (1, 1, 1, 1) -> Decision = True
Pereche Independență D: (T4, T5) -> Doar D comută 0->1, forțând decizia 0->1.

Implementare: tests/test_mcdc.py — 5/5 teste trecute cu succes.

Responsabil: Natalia Moroz (10%)

Cerința 5: Mutant de Ordinul 1 Echivalent

🧬 Mutația de Ordinul 1 Aplicată

• Un mutant este de ordinul 1 dacă este generat prin modificarea unei singure instrucțiuni.
• Un mutant este ECHIVALENT dacă produce ieșiri identice cu programul original pe întregul spațiu de intrare.
• Mutația aplicată pe intervalul orelor de vârf:
• Original: is_peak_hours = (8 <= entry_hour <= 17)
• Mutat: is_peak_hours = (7 < entry_hour < 18)
• Operator utilizat: ROR (Relational Operator Replacement) combinat cu ajustarea constantelor de frontieră.

📐 Demonstrație Matematică & Validare

• Parametrul entry_hour este garantat număr întreg (h in Z).
• Pe mulțimea numerelor întregi, avem echivalențele:
• h >= 8 <===> h > 7
• h <= 17 <===> h < 18
• Prin urmare, conjuncția logică:
• (8 <= h <= 17) <===> (7 < h < 18) este o tautologie pe Z.
• Comportamentul extern al funcției este strict identic pentru orice tuplu de intrare posibil.
• Consecință: Niciun test unitar nu poate omorî acest mutant (este imposibil de ucis).
• Validare Empirică: tests/test_equivalent_mutant.py a rulat produsul cartezian pe toate cele 24 de ore și toate rolurile (24 teste exhaustivi) -> 100% identitate.

Responsabil: Victor Mura (15%)

Cerința 6: Mutanți Ne-Echivalenți (Omorât vs. Neomorât)

🎯 Cazul de Testare Ales: test_bva_occupied_exactly_at_quota

Intrări: quota = 10, occupied = 10 (la cotă), hour = 12 (peak), tier = REGULAR, EV = False, allows_overflow = False
Rezultat Așteptat pe Programul Original: AccessDecision.DENIED_QUOTA_EXCEEDED

💥 Mutantul #1: OMORÂT (Killed)

- Mutație ROR: is_within_quota = (occupied <= quota)
- Efect la test: Pentru o = 10, q = 10, expresia 10 <= 10 devine True (în loc de False).
- Rezultat Mutant: Returnează GRANTED_REGULAR.
- Comportament Test: Aserțiunea eșuează imediat: AssertionError: GRANTED_REGULAR != DENIED_QUOTA_EXCEEDED.
- Stare: Mutantul este DETECTAT și OMORÂT.

🛡️ Mutantul #2: SUPRAVIEȚUIT (Survived)

- Mutație Constantă: Prag overflow redus de la 5 la 4 (overflow_count < 4).
- Este ne-echivalent? Da, pentru overflow activ la ora 12 cu diferență 4 produce ieșiri diferite.
- De ce supraviețuiește la acest test? În testul ales, company_allows_overflow = False.
- Expresia se scurtcircuitează (short-circuit) pe primul termen (False and ...). Subexpresia mutată nu este niciodată evaluată!
- Rezultat Mutant: Returnează DENIED_QUOTA_EXCEEDED. Testul trece -> Mutantul SUPRAVIEȚUIEȘTE.

Responsabil: Natalia Moroz (10%)

Cerința 7: Mutații Automate cu Două Unelte (mutmut & cosmic-ray)

📊 Rezultate Experimentale Comparate

Tool 1: mutmut v2.4.4
- 40 mutanți generați în total
- 30 mutanți omorâți (Killed)
- 10 mutanți supraviețuiți (Survived)
- Scors Brut: 75.0% | Scors pe logica internă: 100%

Tool 2: cosmic-ray v8.7.0
- 28 mutanți generați pe AST-ul de decizii
- 24 mutanți omorâți
- 4 mutanți supraviețuiți
- Scors Brut: 85.7% | Scors Ajustat: 96.0%

Eficacitate suite:
- Pct 1 (EP+BVA+CEG): omoară mutanții de flux și limite.
- Pct 4 (MC/DC): omoară 100% mutanții pe decizia compusă EV.

🔍 Analiza Diferențelor de Operatori

De ce diferă numărul de mutanți (40 vs. 28)?
1. mutmut mută literali și declarații de clasă: înlocuiește membrii enum cu None (REGULAR = None) și mută textul din ValueError('Parametri...').
2. cosmic-ray operează strict pe nodurile AST de calcul și logică: ReplaceComparisonOperator (Lt_Eq, Lt_LtE), ReplaceAndWithOr, NumberReplacer.

De ce au supraviețuit 10 mutanți la mutmut?
- Mutanții 1-9 (Enum declarations): Enum-urile sunt rezolvate la import; membrii nefolosiți într-un test specific nu produc erori.
- Mutantul 19 (Mesaj excepție): Testele folosesc pytest.raises(ValueError), verificând clasa excepției, nu șirul exact de caractere.
- Pe logica funcției evaluate_parking_access: Toți cei 30 de mutanți de cod au fost omorâți (100%).

Responsabil: Echipa (Victor & Natalia)

Asumarea Utilizării AI & Concluzii Finale

🤖 Conformitate Ghid: Utilizare Asistență AI

• Conform cerinței din fișa de laborator, utilizarea asistenței AI este documentată transparent în AI_USAGE_LAB.md.
• Instrument utilizat: Antigravity (Gemini 3.8 Flash High) pentru asistență la structurarea codului boilerplate și a tabelelor Markdown.
• Human Ownership & Validare Umană:
• • Toate cazurile de testare au fost calibrate matematic de echipă pentru a satisface cerințele teoretice (V(G)=7, perechi MC/DC).
• • Echipa a validat manual demonstrația de echivalență a mutantului de ordinul 1 pe numere întregi.
• • Toate rezultatele de acoperire și mutații au fost rulate și verificate empiric în terminal.

🏆 Concluzii & Livrabile Proiect

• 78 de teste unitare automate rulate și trecute (100% success rate).
• Acoperire completă a tuturor cerințelor 1-7 din ghidul de laborator.
• Repartizare echitabilă și clară: Victor Mura 50% | Natalia Moroz 50%.
• Repository Git dedicat pe GitHub sincronizat via HTTPS cu istoricul separat pe fiecare autor: https://github.com/victormura/tss
• Documentație completă cu rapoarte separate pentru fiecare cerință în directorul proiect_laborator/docs/.
•
• Vă mulțumim pentru atenție! • Întrebări & Răspunsuri (Q&A)